FPGA Implementation of a Median Filter

Gavin L. Bates & Saeid Nooshabadi

School of Electrical Engineering,
Northern Territory University, N.T. 0909
gavster@it.ntu.edu.au, saeid@it.ntu.edu.au

ABSTRACT

The median filter is an effective device for the removal of impulse-based noise on video signals. This is due to the partial averaging effect of the median filter and its biasing of the input stream, rather than straight mathematical averaging. In this paper, we describe three realizations of median filter, built into as few as one Field programmable logic device, which is capable of processing an incoming video data stream at a maximum (programmable logic device partially dependent) of around 30 MS/s.

In total, four designs are considered, with a primary design, two variations on the primary design and an asynchronous version based on the primary design. Simulation of the primary design (both synchronous and asynchronous) has demonstrated its potential for reducing the area requirements of a median filter whilst not sacrificing either speed or accuracy.

1. INTRODUCTION

Whilst straight mathematical averaging has proved useful in providing noise reduction in two dimensional signals (such as video imaging), median filtering is recognised as a more effective method of achieving noise reduction[1-3]. This is due to the median filter's superior ability to remove certain types of regularly encountered noise (such as impulse noise) [4] as well as creating less distortion in the output signal than does the straight averaging filter. Median filtering, however is more expensive in terms of size and complexity of chip design.

Traditional median filters use a bubble sort approach [4-6], with an array of 32 dual input sorters, capable of "bubbling" the largest values from the lowest input to the highest output, and consequently, sorting the entire 3x3 matrix into order.

The designs in this paper are based upon triple input sorters (which are in turn, based on dual input sorters), and are not concerned with ordering the entire matrix correctly. In fact, there are only three results guaranteed to be placed correctly in the matrix, those being the maximum, minimum, and the median value of the matrix. Use of this algorithm results in great logic saving when compared with implementation using the bubble sorter algorithm. Although the algorithm presented here does not order the entire matrix correctly, it does produce the identical median value as in the traditional bubble sorter algorithm.

The system based on this sorting technique contains a total of 15 dual input sorters, two single stage storage elements and two scan line FIFO buffers (which includes the logic required for handling serially natured two dimensional signals). On the other hand a traditional bobble sorter system requires 32 dual input sorters, plus additional logic for handling a serially natured two dimensional signal as required.

2. DESCRIPTION OF DESIGNS

The architecture of the device uses a series of reordering blocks to place the input data stream (from both the input source and from the internal FIFO line buffers) into the correct order. The configuration of these filters and the addition of some averaging logic blocks separates the first design from the two others. All three designs consist of at least three sorting stages, containing pipeline buffers between these sorting stages to allow the processing of one 3x3 block of input data every clock cycle. Each variation also contains two small on chip line buffers for remembering the previous two video lines. In larger system implementation, the on chip FIFO buffers can be replaced by larger off chip FIFO buffers.

2.1 Module Descriptions

The following describes the construction of the various modules inside the median filter.

2.1.1 Dual Input Sorter

The dual input sorter, as seen in Figure 1, is made up from two sub-modules, a magnitude detector and an output swapper.

The magnitude detector generates a single output bit, indicating that the value placed on the upper input is smaller than the value placed on the lower input.

The output swapper therefore has a single bit control input and uses standard logic to propagate the input signals as applied, or with the outputs reversed. Note, that the upper or the right-most output of a sorter in Figure 1 is the maximum output.

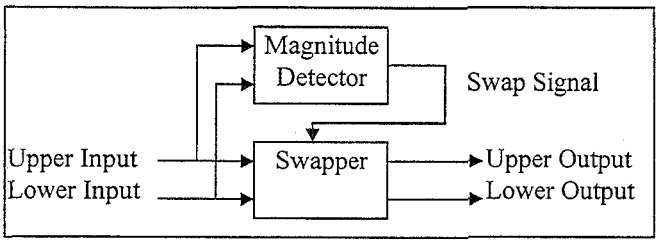

Figure 1: Block Diagram of the Dual Input Sorter.

2.1.2 Triple Input Sorter

The triple input sorter, shown in Figure 2, is built from three dual input sorters, along with pipelining buffers between each dual input sorter. This allows the smallest possible clock cycle.

2.1.3 Averager

The averaging blocks consist of two simple parts; an adder and a 2^{-1} shifter. The adder was constructed using logic gates, whilst the shifter was constructed by simple hard wiring techniques.

2.2 Median Filter Design for Algorithm 1

The primary design, based on algorithm 1, contains no averaging logic blocks and is mostly made up from triple input sorting blocks. Figure 3 describes the working of the algorithm.

The algorithm used in this version of the median filter produces the same result as using a standard 9 input bubble filter block. Although the value of the outer cells in the 3 x 3 matrix in the algorithm described in Figure 3 differ from the corresponding cell values in the standard bubble sorter algorithm, the only one that is important, the median (the middle cell) number, was found to always be that same with both algorithms. This was checked by the creation of a C program, which set up a 3x3 matrix, ran both the algorithms on the raw 3x3 matrix and compared the results. There were no differences between the median number generated by each algorithm for any of the complete set of 387 million matrix tested. Implementation using the algorithm presented in Figure 3 has the advantage of simplicity and lower hardware complexity.

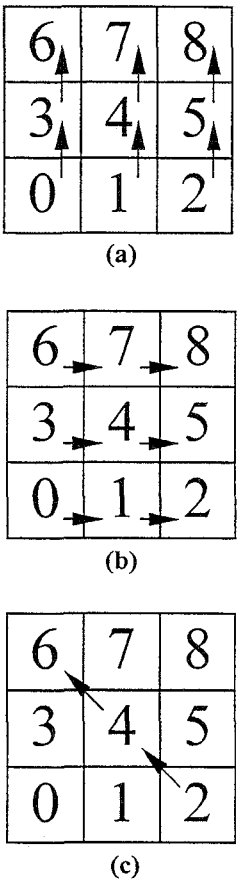

Figure 3: Algorithm 1 (a) Vertical Sorting, (b) Horizontal Sorting and (c) Diagonal Sorting

The algorithm used consists of sorting the columns, followed by sorting the rows then finally a single sort on the diagonal with the largest numbers always being pushed towards the largest numbered cells. This process requires at most 7 triple input sorters, but this is reduced to 5 by storing the results from the initial column sorter and re-using them in the subsequent two calculations.

The block diagram in Figure 4 shows how pipeline registers are used to replace two of the initial sorters, by simply storing the result from the primary input sorter and reusing it later. This can be done due to the serial nature of the input of a video stream and the single pixel movement of the 3x3 median filter window.

The design was created using VHDL in MAX PLUS II from Altera. Simulation indicates that the circuit is capable of running beyond 27 MHz on an Altera

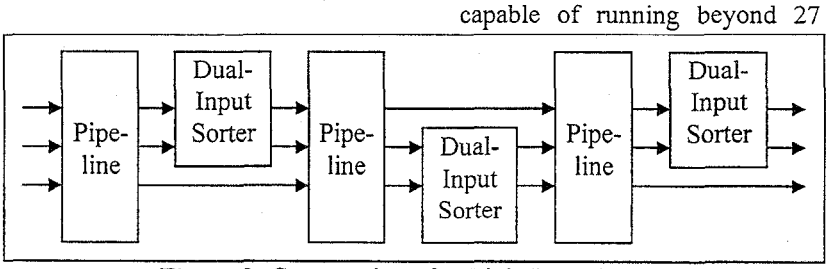

Figure 2: Construction of a Triple Input Sorter.

EPF8820 FPGA, with an output byte on every clock cycle, after an initial 11 cycle startup latency. Compilation into FLEX EPF10K20 FPGA, achieves a performance of 31 MHz.

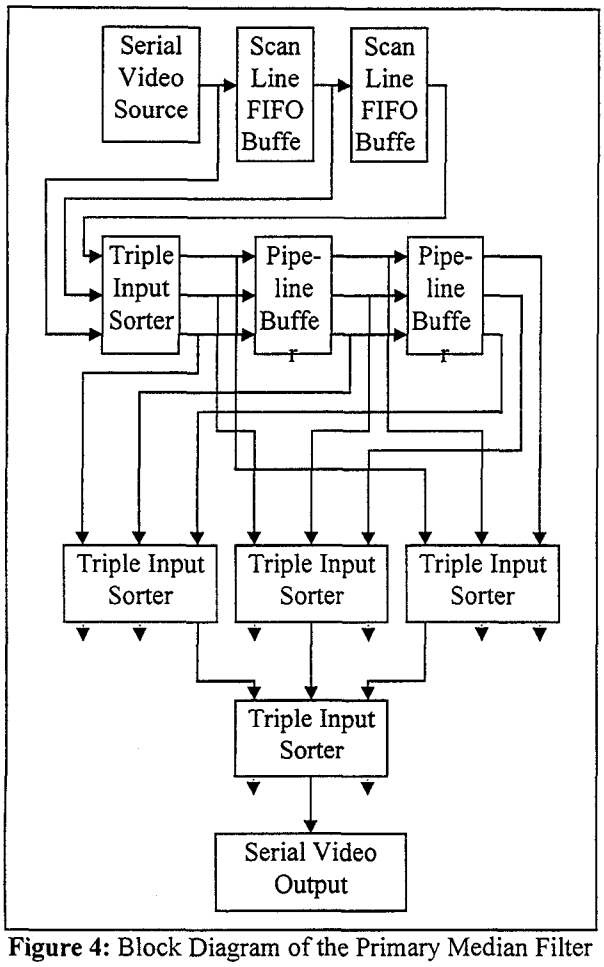

Figure 4: Block Diagram of the Primary Median Filter Design.

The use of this design provides the ability to shrink the active area required for a median filter by a factor of around two.

2.3 Variations on the Primary Design

Another two designs (based on the primary design) were also investigated. Their function was to reduce the space requirements of the median filter, with a nominal loss of accuracy by using averaging logic to reduce the number of cells being sorted.

2.3.1 Variation 1

The second algorithm uses some averaging blocks to reduce the size of the 3x3 matrix to a 2x3 matrix and thus allowing the use of more simple hardware blocks in the design. Figure 5 describes the working of the second algorithm.

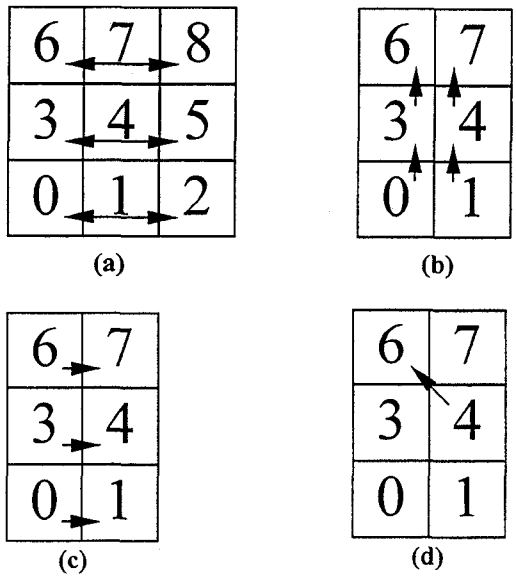

Figure 5: Algorithm 2 (a) Averaging, (b) Vertical Sorting, (c) Horizontal Sorting, and (d) Diagonal Sorting

In these median filters, a triple input sorter is created by serialising three dual input sorters. Thus anywhere a dual input sorter can be used in place of a triple input sorter represents around a 66% space saving.

The averaging in this design is done on the "outside" cells, averaging the values in cells 6 and 8, placing the result in cell 6; averaging the values in cells 3 and 5, placing the result in cell 3 and averaging the values in cells 0 and 2, placing the result in cell 0.

This design was compiled into an Altera FLEX EPF8636 chip but suffered from clock skew problems.

The design was the recompiled to fit into an Altera FLEX EPF10K10 and achieved a performance in excess of 48 MHz .

2.3.2 Variation 2

The third algorithm uses an additional averaging block to reduce by one the size of the matrix to be filtered. Figure 6 describes the working of this algorithm.

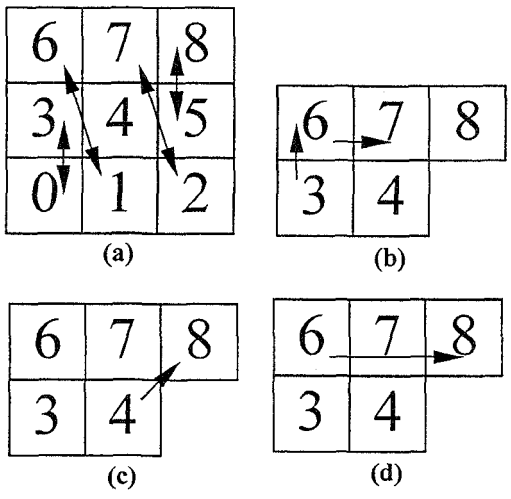

Figure 6: Algorithm 3 (a) Averaging, (b) Residual Sorting, (c) Extra Sorting, and (d) Extra Sorting

The result of the averaging leaves a 2x3 matrix, with no value in the bottom-right. This is done by averaging the results in cells 0 and 3, 1 and 6, 2 and 7, 5 and 8, placing each result of the averaging in a cell relative to their original position around cell 4.

This design also compiled into an Altera FLEX EPF8636 chip and also suffered from clock skew problems.

When recompiled into a FLEX EPF10K10, the clock skew problems disappeared and the system gained a performance of around 48 MHz.

2.4 Asynchronous Design

An asynchronous version of the standard median filter was also created, using the structures documented in [7] which based on Sutherland Micropipelines [8]. This was also created in MAX PLUS II, using the graphical environment.

This design displays the asynchronous nature of the system, by first accepting data at a very fast rate, then suddenly slowing down, as the data backs-up behind slower blocks. The preliminary results obtained from this circuit suggest similar results as obtained from the synchronous standard design.

It should be emphasised that synchronous FPGA is not a suitable medium for the asynchronous designs [7].

3. CONCLUSION

This paper has discussed four variations on the two themes, synchronous and asynchronous median filters. It would appear that the non-averaging 3x3 synchronous median filter solution offers the best opportunity for development of a commercially viable system to reduce noise in a two dimensional signal.

The asynchronous model also shows much promise, however more research is yet to be done, especially into the viability of mass production of asynchronous or micropipelined digital circuitry. From the results, it would appear that the asynchronous approach may also be more appropriate to variable data-rate input streams, due to it's elastic nature.

ACKNOWLEDGEMENT

We appreciate the financial assistance provided by Mr. W. Jessup, the Dean, Faculty of Technology, Northern Territory University.

REFERENCES

[1] T. S. Huang, G. J. Yang and C. Y. Tang, "A fast two-dimensional median filtering algorithm", IEEE Trans. Acoustic Speech and Signal Processing, ASSP-28, pp. 415-421, Aug. 1980.

[2] M O. Ahmed and D. Sundararajan, "A fast algorithm for two-dimensional median filtering", IEEE Trans. On Circuits and Systems, CAS-34, pp. 1364-1375, Nov. 1987.

[3] J. T. Astolla and G. Campbell, "On computing of running median", IEEE Trans. Acoustic Speech and Signal Processing, ASSP-37, pp.572-574, Apr. 1989.

[4] L. A Christopher, W. T. Mayweather and S. S. Perlman, "VLSI median filter for impulse noise elimination in composite or component TV signals", IEEE Trans. on Consumer Electronics, vol 34, no. 1, pp. 263-267, 1988.

[5] M. Karaman, L. Onural and A. Atalar, "Design and implementation of a general purpose median filter in VLSI", VLSI Signal Processing III, IEEE Press, pp. 111-119, 1988.

[6] K. Olfazer, "Design and implementation of a single chip 1-D median filter", IEEE Trans. Acoustic Speech and Signal Processing, ASSP-31, pp. 1164-1168, Oct. 1983.

[7] M. Gamble, B. Rahardjo and R.D. McLeod, "Reconfigurable FPGA Micropipelines", Technical report, U. of Manitoba, 1994.

[8] I. E. Sutherland, "Micropipelines", Communications of ACM, vol. 38, no. 6, 1989